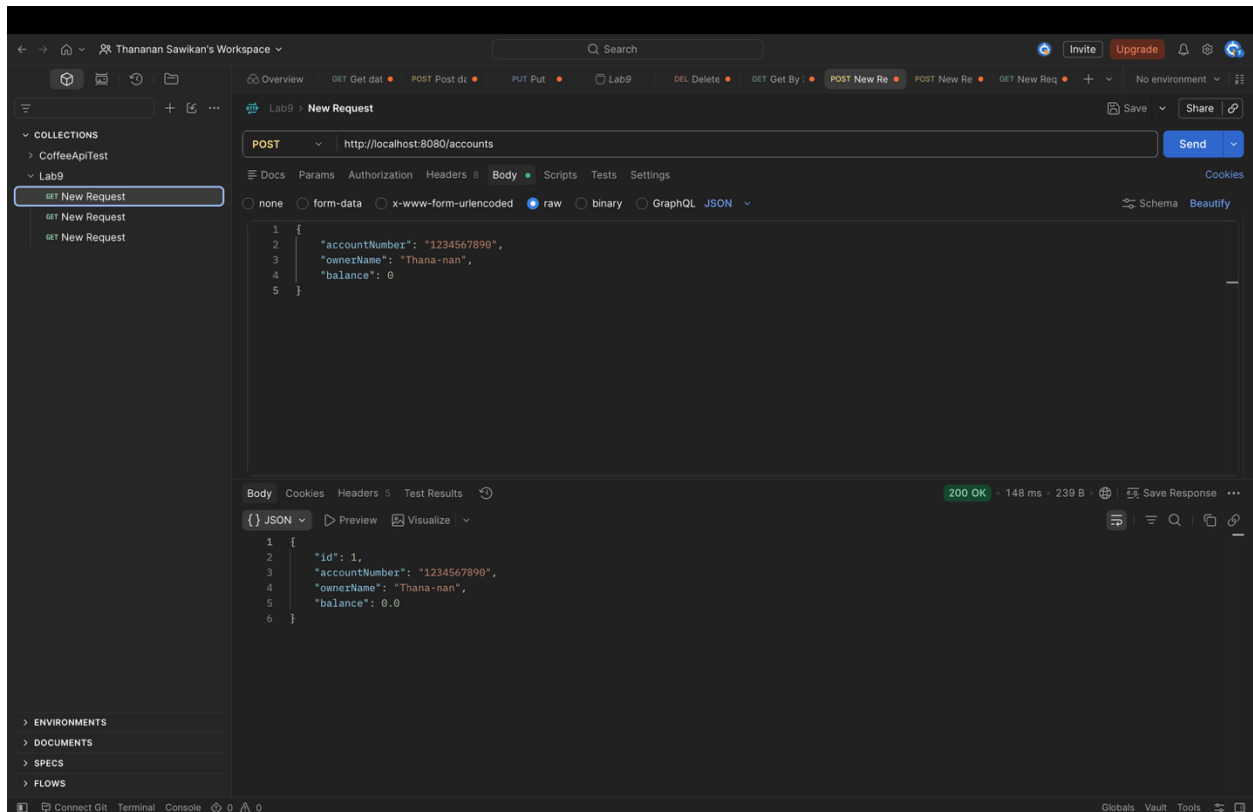


Lab 9 Transactional

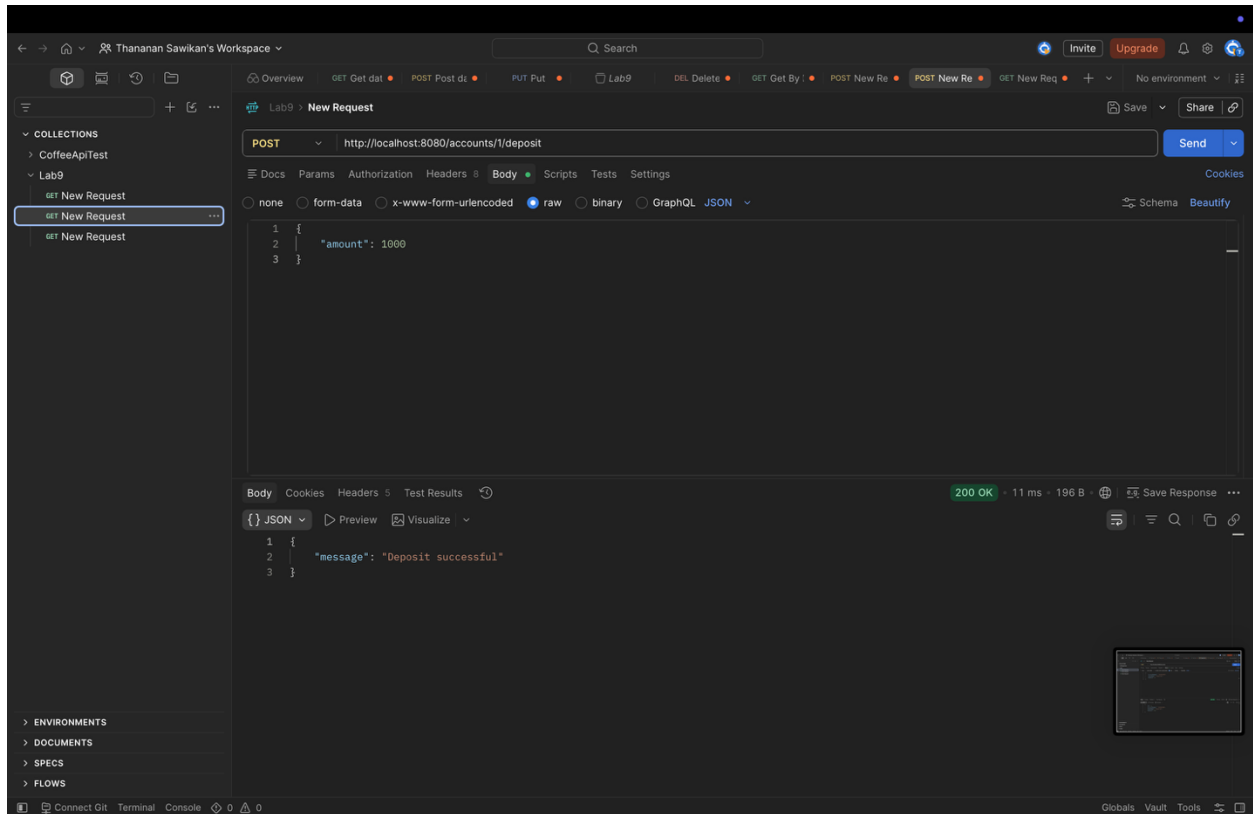
นาย ธนันต์ สาวิกัน 673380586-8

1. การสร้าง Account



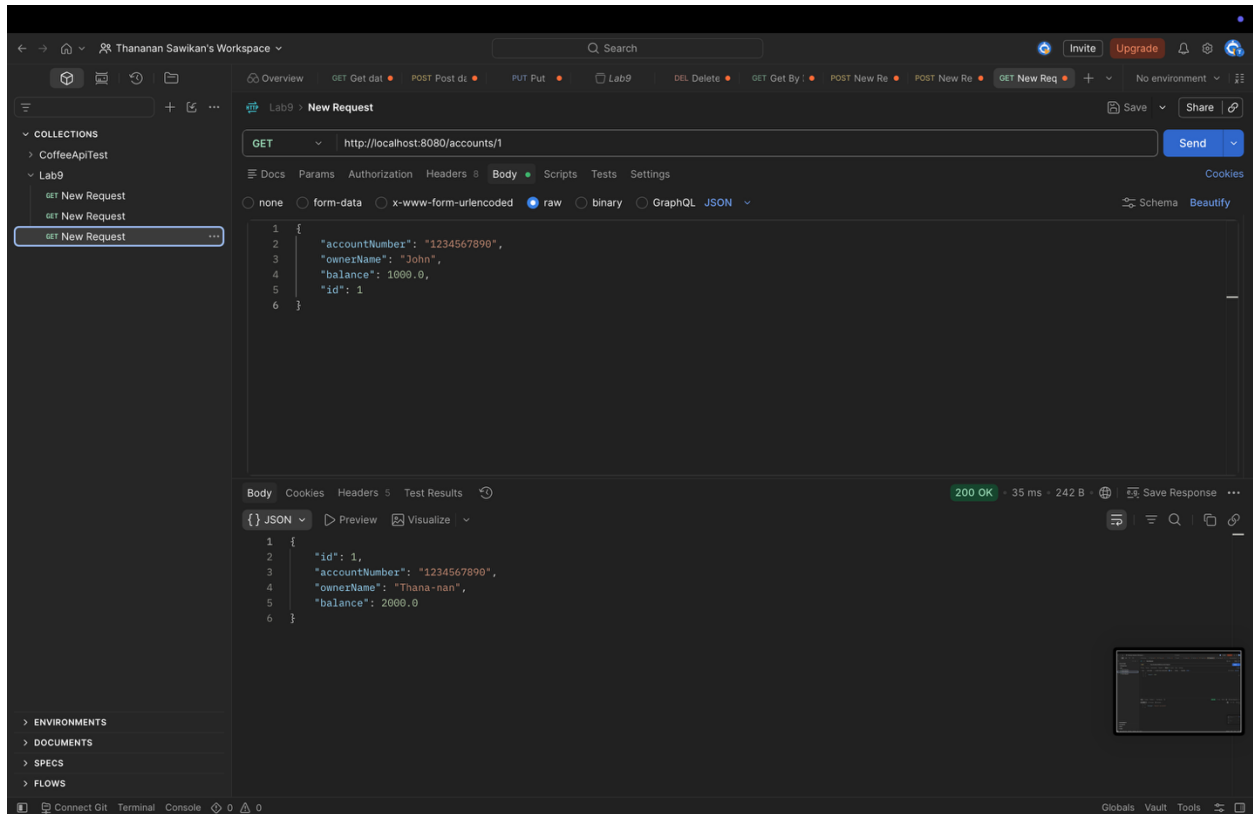
อธิบายผล: ส่ง POST http://localhost:8080/accounts พร้อม body ที่ระบุ accountNumber, ownerName และ balance เริ่มต้นเป็น 0 ระบบตอบกลับด้วย status 200 OK และคืนข้อมูลบัญชีที่ถูกสร้าง พร้อม id: 1 ที่ฐานข้อมูลกำหนดให้อัตโนมัติ (auto-generated primary key) แสดงว่า AccountService.createAccount() ทำงานถูกต้องและบันทึกข้อมูลลงตาราง account สำเร็จ

2. การฝากเงินสำเร็จ



อธิบายผล: ส่ง POST `http://localhost:8080/accounts/1/deposit` พร้อม body `{"amount": 1000}` ระบบตอบกลับ 200 OK พร้อมข้อความ "Deposit successful" แสดงว่าเมธอด `deposit()` ใน `DepositService` ทำงานครบทั้ง 3 ขั้นตอนสำเร็จ คือ ค้นหาบัญชี, เพิ่มยอดเงินเข้า `balance`, และบันทึก `DepositTransaction` ใหม่ลงฐานข้อมูล โดยทั้งหมดอยู่ภายใต้ `Transaction` เดียวกันเพราะมี `@Transactional` กำกับอยู่

3. ตรวจสอบ Account



อธิบายผล: ส่ง GET `http://localhost:8080/accounts/1` เพื่อตรวจสอบว่ายอดเงินเปลี่ยนแปลงจริงหลังฝากเงิน ผลลัพธ์แสดง `balance: 2000.0` ซึ่งยืนยันว่าการฝากเงินถูกบันทึกลงฐานข้อมูลจริง ไม่ใช่แค่ค่าที่ส่งกลับมาชั่วคราวในตอน request ก่อนหน้า

4. ตรวจสอบ DepositTransaction

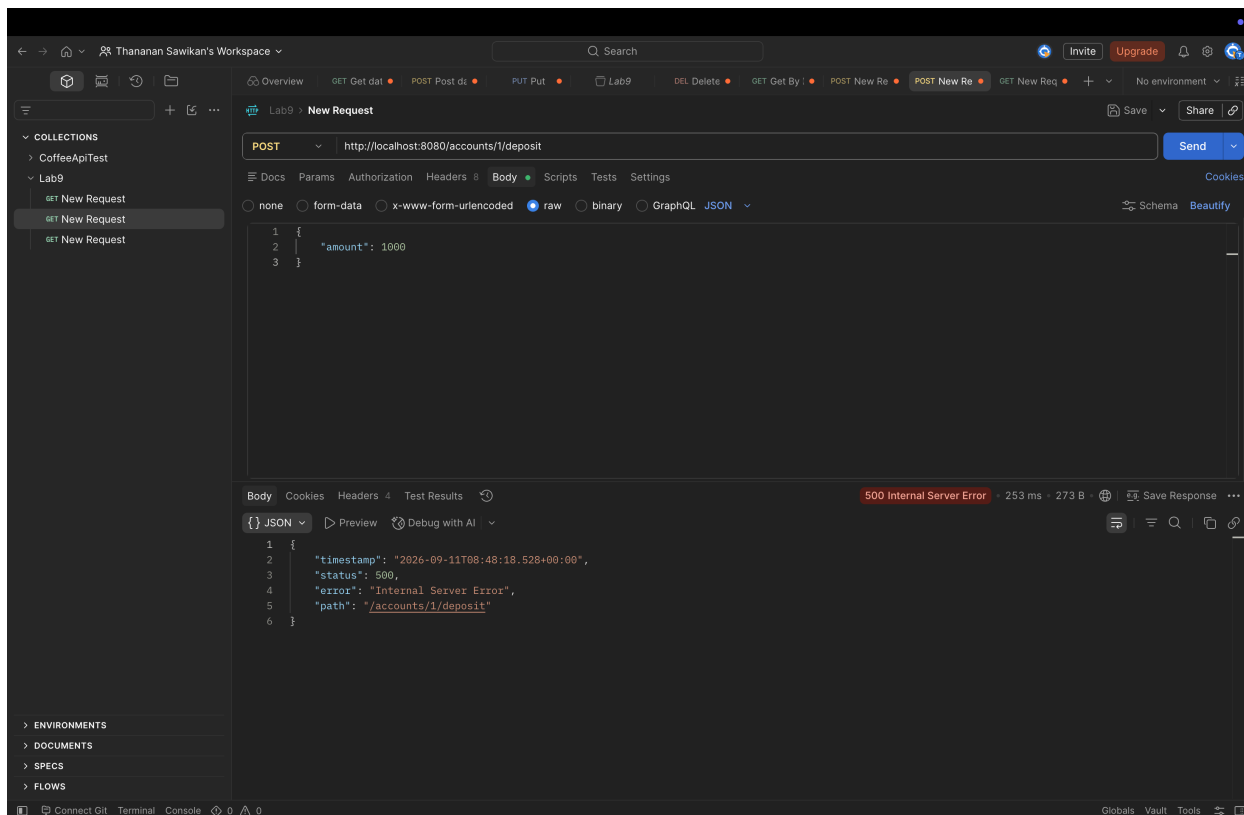
```
postgres=# \c lab9
[psql (18.4, server 16.15 (Homebrew))
You are now connected to database "lab9" as user "tzoey".
lab9=# SELECT * FROM deposit_transaction;
 id | amount | account_id
-----+-----+-----
  1 |   1000 |          1
  2 |   1000 |          1
(2 rows)

lab9=#
```

อธิบายผล: เชื่อมต่อเข้าฐานข้อมูล lab9 ผ่าน psql ด้วยคำสั่ง \c lab9 แล้วรัน SELECT * FROM deposit_transaction; ผลลัพธ์แสดง 2 แถว คือ id 1 และ id 2 โดยทั้งสองแถวมี amount = 1000 และ account_id = 1 เหมือนกัน

ข้อมูลนี้สอดคล้องกับยอด balance = 2000.0 ที่เห็นในข้อ 3 พอตี ยืนยันว่าได้มีการฝากเงินเข้าบัญชีเดียวกัน (id 1) ไปแล้ว 2 ครั้ง ครั้งละ 1000 บาท และทุกครั้งที่ฝากเงินสำเร็จ ระบบบันทึกประวัติลงตาราง deposit_transaction ควบคู่ไปกับการเพิ่ม balance ในตาราง account ครบทั้งสองส่วนตามที่ @Transactional ควบคุมไว้

5. ทดลอง Rollback

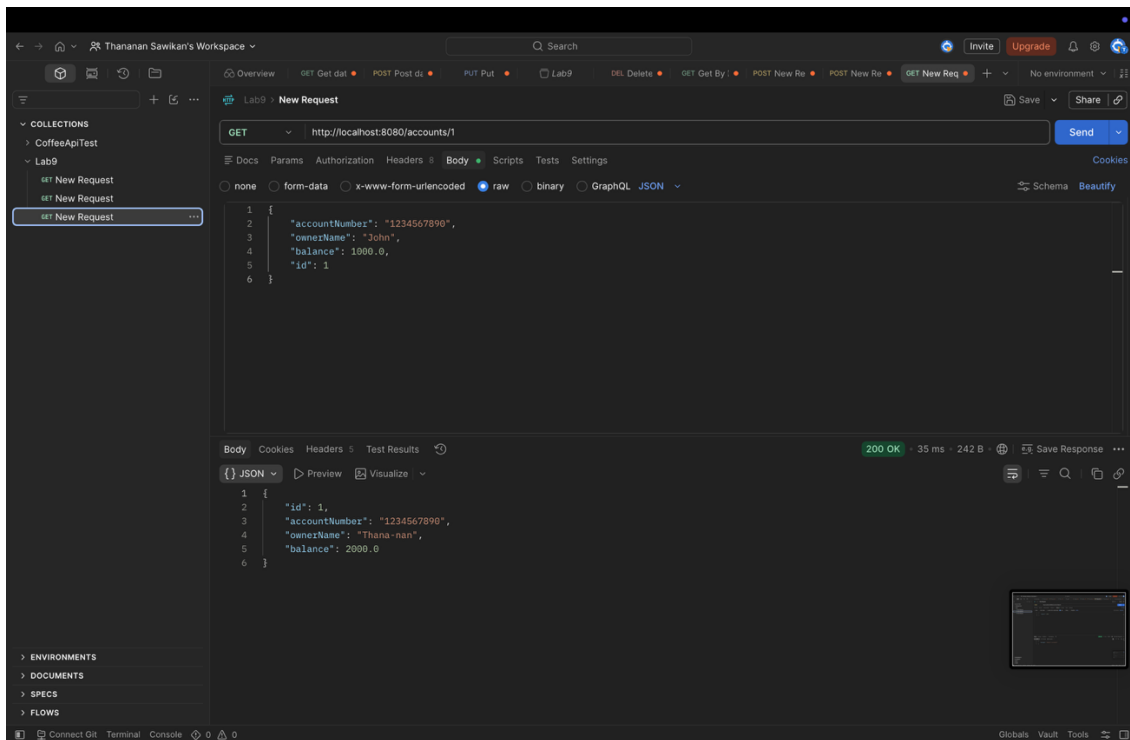
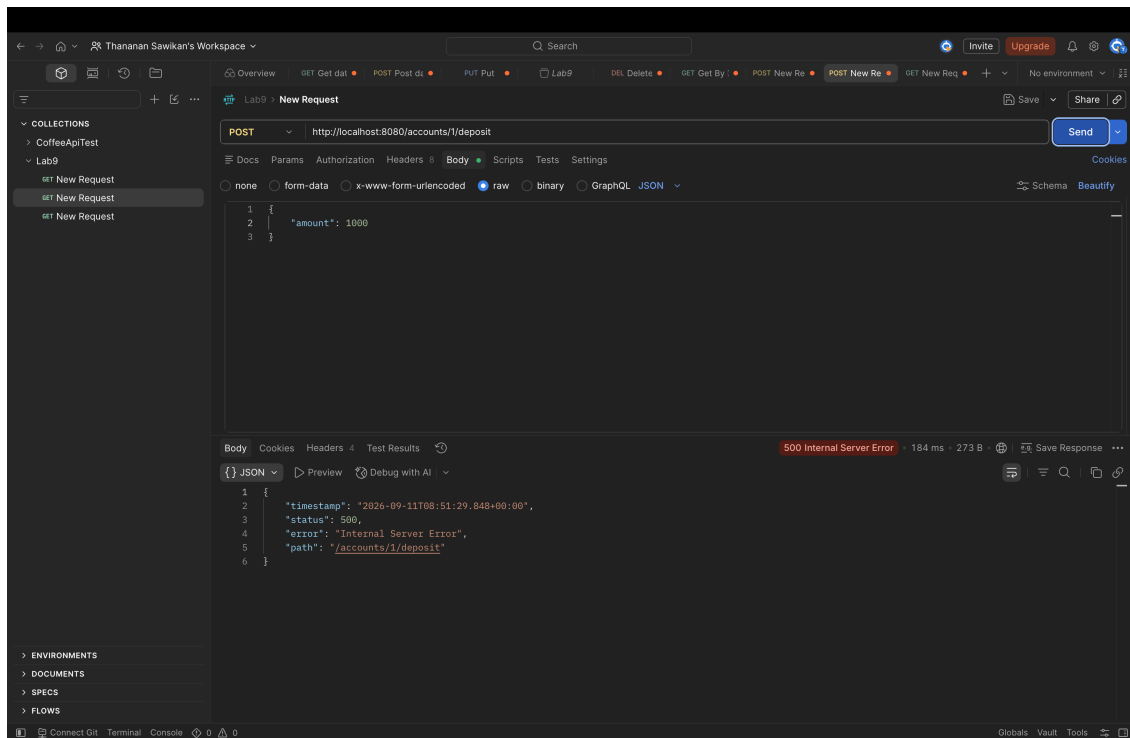


อธิบายผล:

หลังจากเพิ่มบรรทัด `throw new RuntimeException("Test Rollback");` ไว้ที่ท้ายเมธอด `deposit()` ในไฟล์ `DepositService.java` (โดยยังคง `@Transactional` ไว้เหมือนเดิม) แล้วหยุดโปรแกรมและรันใหม่ จากนั้นยิง `POST http://localhost:8080/accounts/1/deposit` ขึ้นด้วย body `{"amount": 1000}` เหมือนเดิม

ผลลัพธ์ที่ได้คือ `500 Internal Server Error` แทนที่จะเป็น `200 OK` เหมือนก่อนหน้านี้ พร้อมข้อความ `"error": "Internal Server Error"` ที่ path `/accounts/1/deposit` แสดงให้เห็นว่า exception ถูกโยนออกมาจริงตามที่ตั้งใจไว้ในโค้ด แม้ว่าคำสั่ง `save()` ทั้งสองรายการ (เพิ่ม balance และบันทึก `DepositTransaction`) จะถูกเรียกไปแล้วก่อนถึงบรรทัด `throw` — จำลองสถานการณ์ที่งานเกือบเสร็จสมบูรณ์แล้วเกิดพังในขั้นตอนสุดท้ายพอดี ซึ่งเป็นกรณีทดสอบที่สำคัญที่สุดสำหรับพิสูจน์ว่า `Transaction` ทำงานถูกต้องหรือไม่

6.เปรียบเทียบมี @Transactional กับไม่มี @Transactional



อธิบายผล:

จากการทดลองทั้งสองกรณี ผัง Client ที่ใช้ Postman ยิง POST /accounts/1/deposit เหมือนกันทุกประการ (มี throw new RuntimeException("Test Rollback") อยู่ในโค้ดทั้งคู่) จะได้รับ 500 Internal Server Error เหมือนกันไม่ต่างกัน เพราะ exception ยังคงถูกโยนออกมาเช่นเดิม ไม่ว่าจะ มี @Transactional หรือไม่ก็ตาม — ความแตกต่างที่แท้จริงจึงไม่ได้อยู่ที่ response ที่ผู้ใช้เห็น แต่อยู่ที่ สถานะของข้อมูลในฐานข้อมูลหลังเกิด error

ตอนมี @Transactional: แม้โค้ดจะสั่ง save() ทั้งสองคำสั่ง (เพิ่ม balance และบันทึก DepositTransaction) ไปแล้วก่อนเกิด error แต่ Spring มองว่าทุกคำสั่งในเมธอด deposit() เป็นทรานแซกชันเดียวกัน เมื่อตรวจพบ RuntimeException ระบบจึงสั่ง ROLLBACK ย้อนการเปลี่ยนแปลงทั้งหมดกลับไปเป็นสถานะก่อนเริ่มทำรายการ ผลคือ balance ในบัญชีและจำนวนแถวในตาราง deposit_transaction ไม่เปลี่ยนแปลงเลยแม้แต่น้อย

ตอนลบ @Transactional ออก: ไม่มีขอบเขตทรานแซกชันมาครอบคำสั่งทั้งสองไว้ด้วยกันอีกต่อไป แต่ละคำสั่ง save() จึงทำงานและ commit ลงฐานข้อมูลทันทีที่ละคำสั่งอย่างเป็นอิสระจากกัน เมื่อ error เกิดขึ้นหลังจาก save() ทั้งสองคำสั่งถูกเรียกไปเรียบร้อยแล้ว ข้อมูลที่บันทึกไปก่อนหน้านี้จึงไม่มีอะไรมาย้อนกลับให้ ผลคือ balance เพิ่มขึ้นจริงและมีแถวใหม่ถูกเพิ่มเข้าไปในตาราง deposit_transaction จริง ทั้งที่ request นั้นจบลงด้วย error

ตอบคำถามท้าย Lab

1. @Transactional มีหน้าที่อะไร?

กำหนดขอบเขตของ transaction ให้ทุกคำสั่งที่เข้าถึงฐานข้อมูลภายในเมธอดนั้นถูกนับเป็นหน่วยงานเดียวกัน (atomic unit) ถ้าทำงานจนจบโดยไม่มี exception ระบบจะ COMMIT ให้ทั้งหมด แต่ถ้าเกิด exception ระหว่างทาง ระบบจะ ROLLBACK ย้อนการเปลี่ยนแปลงทั้งหมดกลับไปเหมือนไม่มีอะไรเกิดขึ้น

2. เพราะเหตุใดการฝากเงินจึงควรใช้ Transaction?

เพราะการฝากเงิน 1 ครั้งต้องเปลี่ยนแปลงข้อมูล 2 จุดพร้อมกัน คือเพิ่ม balance ในบัญชี และบันทึกประวัติใน DepositTransaction ถ้าไม่มี Transaction ควบคุม แล้วเกิด error ระหว่างสองขั้นตอนนี้ (เช่น บันทึก balance สำเร็จแต่บันทึกประวัติล้มเหลว) ข้อมูลจะไม่ตรงกัน (inconsistent) เช่น ยอดเงินเพิ่มแต่ไม่มีประวัติรองรับ ซึ่งเป็นอันตรายมากสำหรับระบบการเงิน

3. COMMIT และ ROLLBACK ต่างกันอย่างไร?

COMMIT คือการยืนยันบันทึกการเปลี่ยนแปลงทั้งหมดในทรานแซกชันนั้นลงฐานข้อมูลอย่างถาวร เกิดขึ้นเมื่อทุกคำสั่งในทรานแซกชันทำงานสำเร็จโดยไม่มีข้อผิดพลาด ส่วน ROLLBACK คือการยกเลิกและย้อนการเปลี่ยนแปลงทั้งหมดกลับไปสู่สถานะก่อนเริ่มทรานแซกชัน เกิดขึ้นเมื่อมีข้อผิดพลาดระหว่างทางทำให้ทรานแซกชันไม่สามารถทำงานจบได้

4. Account และ DepositTransaction มีความสัมพันธ์แบบใด?

เป็นความสัมพันธ์แบบ **One-to-Many** (1 Account มีได้หลาย DepositTransaction) โดยฝั่ง DepositTransaction เป็นผู้เก็บ Foreign Key account_id ผ่าน @ManyToOne ที่ชี้กลับไปยัง Account ส่วนฝั่ง Account ไม่มีฟิลด์เก็บลิสต์ของ DepositTransaction เพื่อป้องกันปัญหการวนอ้างอิงกันไม่รู้จบ (infinite loop) ตอนแปลงข้อมูลเป็น JSON

5. เมื่อเกิด Error ขณะใช้ @Transactional ข้อมูลใน Database เปลี่ยนแปลงอย่างไร?

ไม่มีข้อมูลใดถูกเปลี่ยนแปลงหรือบันทึกลงฐานข้อมูลเลย แม้ว่าโค้ดจะเรียก save() ไปแล้วทั้งสองคำสั่งก่อนเกิด error ก็ตาม เพราะ Spring จะสั่ง ROLLBACK ทรานแซกชันทั้งก้อนทันทีที่ตรวจพบ exception ทำให้ balance และตาราง deposit_transaction กลับสู่สถานะเดิมก่อนเริ่มทำการ

6. จากการทดลอง เมื่อลบ @Transactional ออก ผลลัพธ์แตกต่างจากเดิมอย่างไร?

เมื่อไม่มี @Transactional แต่ละคำสั่ง save() จะทำงานและ commit ลงฐานข้อมูลทันทีที่คำสั่งอย่างอิสระ ไม่มีการมองภาพรวมของงานทั้งก้อน ดังนั้นเมื่อเกิด error หลังจาก save() ทั้งสองคำสั่งถูกเรียกไปแล้ว ข้อมูลที่บันทึกไปก่อนหน้านี้ (ทั้ง balance ที่เพิ่มขึ้นและแถวใหม่ในตาราง deposit_transaction) จะยังคงค้างอยู่ในฐานข้อมูลถาวร ไม่มีการย้อนกลับ ทำให้ข้อมูลไม่สอดคล้องกัน (data inconsistency) ต่างจากตอนมี @Transactional ที่ข้อมูลกลับสู่สถานะเดิมปะ